

Datastructuren

Authors

Matt ter Steege
Matttersteege@gmail.com

Contents

1	Invariant & binair zoeken	3
1.1	Binary search	3
2	Sommaties & Logaritmen	3
2.1	Rekenkundige reeks	4
2.2	Meetkundige reeks	4
2.3	Harmonische reeks	4
2.4	Rekenregels voor sommaties	5
2.5	Belangrijke eigenschappen	5
3	Analyse en de Grote O	5
3.1	Exacte Looptijdanalyse	5
3.2	De Grote O Notatie	6
3.3	Complexiteitsklassen	6
3.4	Variaties op de O	6
4	Recursie	6
4.1	Wat is Recursie?	6
4.2	Verdeel en Heers Aanpak	6
4.3	Voorbeelden	6
4.4	Correctheid Bewijzen	7
4.5	Recurrente Betrekkingen	7
5	Recursieve Algoritmen Analyseren	7
5.1	Recurrente Betrekkingen Oplossen	7
5.2	Asymptotisch Oplossen	7
5.3	Substitutiemethode	7
5.4	Masterstelling	8
6	Kansrekening	8
6.1	Combinatoriek	8
7	Quick sort	8
8	Linear sorteren	10
8.1	Counting Sort	10
8.2	Geheugengebruik	10
8.3	Radix Sort	10
8.4	Bucket Sort	11
8.5	Ondergrens voor sorteren	11
8.6	Conclusie	11

subsection.9.1subsection.9.3subsection.9.4subsection.9.5subsubsection.9.6.1section.1

9	kansproblemen	12
9.1	Voorbeelden van Bernoulli-experimenten	12
9.2	Bernoulli-proces met vaste lengte	12
9.3	Kans op minstens één succes	12
9.4	Aantal successen in n experimenten	13
9.5	Binomiale verdeling	13
9.6	Nuttige formules	14
10	Abstracte datastructuren	14
11	Stack en Queue	15
11.1	De stack	15
11.2	Queue	15
11.3	Priority queue	16
12	Heap	16
12.1	Heapify	17
12.2	BuildHeap	17
12.3	ExtractMin	17
12.4	Rootify	17
12.5	Insert	17
12.6	DecreaseKey	17
12.7	Heapsort	17
13	Hashtabel	18
13.1	Directe adressering	18
13.2	Hashing	18
13.3	Eigenschappen van een goede hashfunctie	18
13.4	Vergelijking: hashtabel vs. zoekboom	18
14	Binaire Zoekbomen	18
14.1	Inorder-doorloop	19
14.2	Zoeken	19
14.3	Minimum, Maximum, Successor en Predecessor	19
14.4	Toevoegen	19
14.5	Verwijderen	19

1 | Invariant & binair zoeken

Een invariant is een logische uitspraak over variabelen die vóór en na elke iteratie van een loop waar blijft. Het doel is om de correctheid van de loop te structureren. Bij het zoeken naar een geschikte invariant werk je vanuit de gewenste eindtoestand. Vervolgens controleer je:

- Initialisatie: kies beginwaarden (bijv. $i = -1$, $j = n$) zodat de invariant waar is.
- Body: pas i of j zo aan dat de invariant behouden blijft.
- Conditie: stop wanneer $i = j - 1$, zodat invariant $\wedge \neg C$ de gewenste eigenschap geeft.

Daarnaast gebruik je een variant (bijv. $j - i$) die strikt afneemt om terminatie te garanderen.

Voorbeeld (**invariant vinden**):

Gegeven een gesorteerde array A en een waarde x . We willen de eerste index j vinden waarvoor $A[j] \geq x$.

Stap 1: Einddoel formuleren

We willen uiteindelijk:

$$A[j-1] < x \quad \wedge \quad A[j] \geq x$$

Stap 2: Verzwakken naar een invariant

Tijdens de loop is $j - 1$ nog niet vast. Daarom nemen we een algemene vorm:

$$A[i] < x \quad \wedge \quad A[j] \geq x$$

Stap 3: Initialisatie kiezen

Kies $i = -1$ en $j = n$. Met de conventie $A[-1] = -\infty$ en $A[n] = +\infty$ geldt de invariant direct.

Stap 4: Body afleiden

Neem $m = \lfloor (i + j)/2 \rfloor$ ¹.

- Als $A[m] < x$, zet $i = m$
- Anders zet $j = m$

In beide gevallen blijft de invariant behouden.

Stap 5: Stopconditie

Stop als $i = j - 1$. Dan volgt uit de invariant precies de gewenste eigenschap.

Zoeken

Zoeken in een gesorteerde array maakt gebruik van orde: vergelijkingen laten toe om delen van de array uit te sluiten in plaats van lineair te doorlopen.

Binary search

Binary search houdt een interval $[i, j]$ bij waarin de oplossing ligt.

Per iteratie:

- neem $m = \lfloor (i + j)/2 \rfloor$
- als $A[m] < x$, zet $i = m$
- anders zet $j = m$

De invariant garandeert dat de oplossing binnen $[i, j]$ blijft. De variant $j - i$ halveert ongeveer elke stap, wat leidt tot $O(\log n)$ tijd.

2 | Sommaties & Logaritmen

Een sommatie heeft de vorm:

$$\sum_{i=p}^q A_i$$

waarbij:

- i : de sommatievariabele (gebonden, mag hernoemd worden)

¹Dit is het binary search algoritme

- p, q : onder- en bovengrens (samen het domein)
- A_i : de termformule, afhankelijk van i

Rekenkundige reeks

Een **rekenkundige rij** heeft een constant verschil tussen opeenvolgende termen. Een rekenkundige reeks is de sommatie van zo'n rij.

$$\sum_{i=p}^q A_i = \frac{\text{Aantal} \times (\text{Eerste} + \text{Laatste})}{2}$$

Afleiding: Stel $S = \sum_{i=0}^n i$. Optellen van S en S omgekeerd:

$$\begin{aligned} S &= 0 + 1 + 2 + \dots + n \\ S &= n + (n-1) + (n-2) + \dots + 0 \\ 2S &= n + n + n + \dots + n = (n+1) \cdot n \end{aligned}$$

Dus: $S = \frac{n(n+1)}{2}$.

Meetkundige reeks

Een **meetkundige rij** heeft een constante verhouding (groeifactor r) tussen opeenvolgende termen. De algemene vorm is:

$$\begin{aligned} \sum_{i=0}^{n-1} A \cdot r^i \\ \sum_{i=0}^{n-1} A \cdot r^i = A \cdot \frac{r^n - 1}{r - 1} = \frac{\text{Volgende} - \text{Eerste}}{\text{Groeifactor} - 1} \end{aligned}$$

Speciaal geval (groeifactor 2):

$$\sum_{i=0}^n 2^i = 2^{n+1} - 1$$

Oneindige meetkundige reeks: Voor $|x| < 1$ geldt:

$$\sum_{i=0}^{\infty} x^i = \frac{1}{1-x}$$

Harmonische reeks

Het **harmonisch getal** H_n is gedefinieerd als:

$$H_n = \sum_{i=1}^n \frac{1}{i} = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n}$$

Er bestaat geen gesloten formule voor H_n , maar wel een schatting:

$$\ln(n+1) < H_n < \ln(n) + 1$$

Oftewel: $H_n \approx \ln n$

Voorbeelden:

$$\begin{aligned} \sum_{i=1}^n \frac{1}{2i} &= \frac{1}{2} H_n \quad (\text{som van even breuken}) \\ \sum_{i=1}^n \frac{1}{2i-1} &= H_{2n} - \frac{1}{2} H_n \quad (\text{som van oneven breuken}) \end{aligned}$$

Rekenregels voor sommaties

$$\begin{aligned}\sum_{i=p}^q C \cdot A_i &= C \cdot \sum_{i=p}^q A_i && \text{(constante factor)} \\ \sum_{i=p}^q C &= (q - p + 1) \cdot C && \text{(constante term)} \\ \sum_{i=p}^q (A_i + B_i) &= \sum_{i=p}^q A_i + \sum_{i=p}^q B_i && \text{(termsplitsing)} \\ \sum_{i=p}^q A_i &= \sum_{i=p}^r A_i + \sum_{i=r+1}^q A_i && \text{(domeinsplitsing)} \\ \sum_{i=p}^q A_i &= \sum_{i=p}^{q-1} A_i + A_q && \text{(afsplitsen)} \\ \sum_{i=p}^q A_i &= \sum_{j=p}^q A_j && \text{(hernoemen)} \\ \sum_{i=p}^q A_i &= \sum_{i=\pi(p)}^{\pi(q)} A_{\pi^{-1}(i)} && \text{(dummy transformatie)}\end{aligned}$$

Let op: Bij dummy transformatie moet π een bijectie zijn!

Logaritmen

De **logaritme** $\log_b a$ is de oplossing in x van de vergelijking:

$$b^x = a$$

Belangrijke eigenschappen

$$\begin{aligned}b^{\log_b a} &= a && \text{(definitie)} \\ \log_b 1 &= 0, \quad \log_b b = 1 \\ \log_b (a_1 \cdot a_2) &= \log_b a_1 + \log_b a_2 && \text{(productregel)} \\ \log_b (x^y) &= y \cdot \log_b x && \text{(machtregel)} \\ \log_b a &= \frac{\log_c a}{\log_c b} && \text{(grondtal conversie)} \\ \log_b a &= -\log_b (1/a) \\ \log_b a &= -\log_{1/b} a\end{aligned}$$

3 | Analyse en de Grote O

Exacte Looptijdanalyse

Bij het analyseren van algoritmen tellen we de uitvoeringstijd per instructie. Voor een **for**-lus met n iteraties:

- Initialisatie: 1 keer
- Test: $(n + 1)$ keer
- Increment: n keer
- Body: n keer

Voorbeeld Selection Sort: Voor geneste loops met variabele iteraties gebruiken we sommaties:

$$T(n) = \sum_{i=0}^{n-1} (n-i)(c_1) = \frac{n(n+1)}{2} \cdot c_1 = An^2 + Bn + C$$

De Grote O Notatie

Definitie: $f(n) = O(g(n))$ betekent: $\exists n_0, c > 0 : \forall n > n_0 : f(n) \leq c \cdot g(n)$

Belangrijke eigenschappen:

- $O(2n^2) = O(n^2)$ (constanten mogen weg)
- $n = O(n^2)$ (kleinere termen worden geabsorbeerd)
- $O(n^2) + O(n) = O(n^2)$
- $O(n^2) + O(n^2) = O(n^2)$

Complexiteitsklassen

In oplopende volgorde van groeisnelheid:

Logaritmisch: $\lg n, \lg^2 n, \lg^k n$

- Grondtal maakt niet uit: $O(\lg n) = O(\ln n) = O(\log n)$
- Macht binnen log: $O(\lg(n^2)) = O(\lg n)$
- Macht van log: $O(\lg^2 n) \neq O(\lg n)$

Polynomiaal: $n, n^2, n^3, \sqrt{n}$

- Hoogste exponent is leidend: $n^2 + 2n^3 = O(n^3)$
- Domineert altijd logaritmisch: $\lg^k n = o(n^\varepsilon)$ voor elke $\varepsilon > 0$

Exponentieel: $2^n, 3^n, 5^{n/2}$

- Grootste grondtal is leidend: $2^n + 3^n = O(3^n)$
- Domineert altijd polynomiaal

Variaties op de O

- $O(g(n))$: hoogstens $g(n)$ (bovengrens)
- $\Omega(g(n))$: minstens $g(n)$ (ondergrens): $\exists n_0, c > 0 : \forall n \geq n_0 : f(n) \geq c \cdot g(n)$
- $\Theta(g(n))$: precies $g(n)$ (strakke grens): $f(n) = O(g(n)) \wedge f(n) = \Omega(g(n))$

4 | Recursie

Wat is Recursie?

Voor programmeurs: Een functie roept zichzelf aan.

Voor algoritmeontwerp: Je neemt aan dat je kleinere instanties van het probleem al kunt oplossen.

Belangrijke principes:

- Focus op *wat* de methode doet, niet *hoe*
- Schrijf een exacte postconditie op
- Probeer nooit recursie "uit te rollen" in je hoofd

Verdeel en Heers Aanpak

1. Breek het probleem op in kleinere deelproblemen
2. Los elk deelprobleem recursief op
3. Combineer deeloplossingen tot totaaloplossing
4. Definieer een basisgeval voor kleine problemen

Voorbeelden

Torens van Hanoi:

```
Hanoi(n, source, dest):  
    if n == 1:  
        Zet(source, dest)
```

```

else:
    Hanoi(n-1, source, temp)
    Zet(source, dest)
    Hanoi(n-1, temp, dest)

```

Merge Sort:

- Verdeel array in twee helften
- Sorteer beide helften recursief
- Vlecht (merge) beide gesorteerde helften
- Merge-stap kost $\Theta(n)$ tijd

Correctheid Bewijzen

Om een recursief algoritme correct te bewijzen (met inductie):

1. **Variant:** Identificeer maat voor voortgang (bijv. y -waarde), toon aan dat deze bij recursie strikt daalt
2. **Geldigheid:** Toon aan dat recursieve aanroepen voldoen aan preconditionie
3. **Basisgeval:** Bewijs dat base case correct is
4. **Recursief geval:** Neem aan dat recursieve aanroep correct is (IH), bewijs dat huidige aanroep correct is

Recurrente Betrekkingen

Een recurrente betrekking (RB) definieert een functie in termen van kleinere parameters.

Voorbeeld - Hanoi zetten:

$$Z(n) = \begin{cases} 1 & \text{als } n = 1 \\ 2 \cdot Z(n-1) + 1 & \text{als } n > 1 \end{cases}$$

Oplossing: $Z(n) = 2^n - 1$ (bewijs met inductie)

5 | Recursieve Algoritmen Analyseren

Recurrente Betrekkingen Oplossen

Merge Sort RB:

$$M(n) = \begin{cases} O(1) & \text{als } n \leq 1 \\ 2 \cdot M\left(\frac{n}{2}\right) + \Theta(n) & \text{als } n > 1 \end{cases}$$

Asymptotisch Oplossen

Bij asymptotische analyse mogen we negeren:

- De randvoorwaarde (meestal niet belangrijk)
- Afronding bij delen ($\lfloor n/2 \rfloor$ vs $n/2$)
- Exacte constanten in extra termen

Wat WEL belangrijk is: Het aantal recursieve aanroepen!

Substitutiemethode

Raad een oplossing en bewijs deze met inductie.

Voorbeeld: Bewijs dat $M(n) \leq c \cdot n \lg n$ voor $M(n) \leq 2M(n/2) + dn$

Bewijs (inductiestap):

$$\begin{aligned}
 M(n) &\leq 2M(n/2) + dn \\
 &\leq 2 \cdot c \frac{n}{2} \lg \frac{n}{2} + dn \quad (\text{IH}) \\
 &= cn(\lg n - 1) + dn \\
 &= cn \lg n - n(c - d) \\
 &\leq cn \lg n \quad \text{als } c \geq d
 \end{aligned}$$

Masterstelling

Voor recurrente betrekkingen van de vorm:

$$T(n) = a \cdot T(n/b) + f(n)$$

Stappenplan:

1. Schrijf $f(n) = n^r \cdot \lg^s n$
2. Bereken $\ell = \log_b a$
3. Vergelijk $f(n)$ met n^ℓ :

Geval 1: $r < \ell$ (polynomiaal kleiner) $\Rightarrow T(n) = \Theta(n^\ell)$

Geval 2: $r = \ell$ en $s = 0$ (evenwicht) $\Rightarrow T(n) = \Theta(n^\ell \lg n)$

Geval 3: $r > \ell$ (polynomiaal groter) $\Rightarrow T(n) = \Theta(f(n))$

Voorbeelden:

- Merge sort: $T(n) = 2T(n/2) + \Theta(n)$, dus $a = 2, b = 2, \ell = 1, r = 1 \Rightarrow T(n) = \Theta(n \lg n)$
- Karatsuba: $T(n) = 3T(n/2) + \Theta(n)$, dus $\ell = \log_2 3 \approx 1.58 \Rightarrow T(n) = \Theta(n^{1.58})$

Totale werk: $T(n) = \sum_{i=0}^{\log_b n} a^i \cdot f(n/b^i)$

6 | Kansrekening

Combinatoriek

Faculteit: $n! = 1 \cdot 2 \cdot 3 \cdots n$ met $0! = 1$

Stirling: $n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$

Permutaties (volgorde belangrijk): $P(n, k) = \frac{n!}{(n-k)!} = n(n-1) \cdots (n-k+1)$

Combinaties (volgorde niet belangrijk): $\binom{n}{k} = \frac{n!}{k!(n-k)!} = \frac{P(n, k)}{k!}$

Symmetrie: $\binom{n}{k} = \binom{n}{n-k}$

Alles hierboven is voor de midterm, onder voor de final

7 | Quick sort

Quicksort is een efficiënt, recursief sorteeralgoritme gebaseerd op het principe van *divide and conquer*. Het algoritme kiest eerst een *pivot* (scharnierpunt) en verdeelt vervolgens de array in twee delen: elementen kleiner dan of gelijk aan de pivot en elementen groter dan de pivot. Dit proces heet de *partition* of *split*. Na de partition staat de pivot direct op zijn definitieve positie in de gesorteerde array.

Vervolgens worden de linker- en rechterhelft onafhankelijk van elkaar recursief gesorteerd. Omdat elementen uit verschillende delen nooit meer met elkaar vergeleken hoeven te worden, wordt het probleem steeds kleiner totdat deelarrays van grootte 0 of 1 overblijven.

De keuze van de pivot heeft grote invloed op de prestaties. In de praktijk wordt vaak een willekeurig element gekozen om extreem scheve verdelingen te voorkomen. Bij een ongelukkige keuze (bijvoorbeeld steeds het grootste of kleinste element) ontstaat een zeer ongebalanceerde verdeling, wat leidt tot een looptijd van $O(n^2)$.

Gemiddeld presteert randomized quicksort echter veel beter. Met een willekeurige pivot wordt de array meestal redelijk evenwichtig opgesplitst, waardoor de verwachte looptijd uitkomt op:

$$O(n \log n).$$

In praktische implementaties wordt quicksort vaak gecombineerd met insertion sort. Wanneer een deelarray kleiner wordt dan een bepaalde drempel (bijvoorbeeld 20 elementen), is insertion sort sneller dan verdere recursie. Deze combinatie levert in de praktijk zeer goede prestaties op.

De belangrijkste eigenschappen van quicksort zijn:

- Gemiddelde looptijd: $O(n \log n)$.
- Slechtste geval: $O(n^2)$.
- Werkt *in-place* met weinig extra geheugen.

-
- Meestal niet stabiel.
 - Zeer geschikt voor grote datasets.
 - De pivot moet random gekozen worden, je kan nooit specifiek kiezen, want dat betekent dat je alle elementen moet doorlopen om een “goede” keuze te maken, wat het algoritme langzamer maakt

Voorbeeld (**Quick sort**):

We willen de volgende array sorteren:

$$A = [8, 2, 1, 5, 8, 7, 3, 2]$$

Kies als pivot het element 3 (Meestal neem je een random element als pivot).

$$[8, 2, 1, 5, 8, 7, \boxed{3}, 2]$$

Na de *partition* worden alle elementen kleiner dan of gelijk aan de pivot links geplaatst, en alle grotere elementen rechts:

$$[2, 1, 2 \mid \boxed{3} \mid 8, 5, 8, 7]$$

De pivot staat nu definitief op zijn juiste positie. We hoeven alleen nog de twee deelarrays te sorteren:

$$[2, 1, 2] \quad \text{en} \quad [8, 5, 8, 7]$$

Linkerdeel

Kies pivot 2 (random):

$$[2, 1, 2]$$

Na partition:

$$[1 \mid \boxed{2} \mid 2]$$

Dit deel is nu volledig gesorteerd:

$$[1, 2, 2]$$

Rechterdeel

Kies pivot 7 (random):

$$[8, 5, 8, 7]$$

Na partition:

$$[5 \mid \boxed{7} \mid 8, 8]$$

Ook hier zijn de resterende deelarrays triviaal:

$$[5] \quad [8, 8]$$

Dus krijgen we:

$$[5, 7, 8, 8]$$

Samenvoegen

Nu combineren we de reeds gesorteerde delen:

$$[1, 2, 2] + [3] + [5, 7, 8, 8]$$

waardoor de uiteindelijke gesorteerde array ontstaat:

$$[1, 2, 2, 3, 5, 7, 8, 8]$$

8 | Linear sorteren

Lineaire sorteeralgoritmen kunnen sneller zijn dan vergelijking-gebaseerde algoritmen zoals QuickSort. Dit kan alleen als er extra informatie bekend is over de structuur van de keys.

Counting Sort

Counting Sort werkt wanneer de keys afkomstig zijn uit een kleine verzameling van M mogelijke waarden.

Voorbeeld:

$$\begin{array}{c|cccccc} i & 0 & 1 & 2 & 3 & 4 & 5 & 6 \\ \hline A[i] & a & d & b & d & b & a & d \end{array}$$

Met keyset $\{a, b, c, d\}$.

We tellen hoe vaak elke key voorkomt in een frequentietabel C :

$$C = [2, 2, 0, 3]$$

Hieruit kunnen we direct de gesorteerde uitvoer construeren:

aabbddd

De tijdscomplexiteit bestaat uit:

- $O(n)$ voor het tellen van de frequenties
- $O(M)$ voor het doorlopen van alle mogelijke keys

Dus:

$$T(n) = O(n + M)$$

Wanneer records naast de key ook extra data bevatten, wordt eerst een cumulatieve tabel berekend. Daarmee kan elk record rechtstreeks op zijn definitieve positie in een nieuwe array geplaatst worden.

Eigenschappen:

- Tijd: $O(n + M)$
- Extra geheugen: $O(M)$ of $O(n + M)$ bij records
- Stabiel sorteeralgoritme
- Niet in-place

Geheugengebruik

Een algoritme heet *in-situ* (of *in-place*) wanneer het weinig extra geheugen gebruikt.

- Wel in-place: QuickSort, HeapSort, InsertionSort
- Niet in-place: Counting Sort, MergeSort, BucketSort

Counting Sort gebruikt een extra frequentietabel en vaak ook een tweede array voor het verplaatsen van records.

Radix Sort

Radix Sort wordt gebruikt wanneer de key uit meerdere kleine onderdelen (*digits*) bestaat.

Voorbeelden:

- Datums
- Telefoonnummers
- Integers

Het algoritme sorteert meerdere keren, telkens op één digit.

Belangrijk: er wordt begonnen met de **minst significante digit** (LSD) en geëindigd met de meest significante digit.

Voor iedere digit wordt een **stabiel** sorteeralgoritme gebruikt, meestal Counting Sort.

Pseudo-code:

```
RadixSort(A,d)
for i = d downto 0
    sorteer stabiel op digit i
```

Eigenschappen:

- Vereist stabiele tussenliggende sorteringen
- Werkt niet correct wanneer je begint met de meest significante digit
- Tijd: $O(d(n + M))$

waarbij d het aantal digits is.

Voor 32-bit integers kan men bijvoorbeeld eerst op de laagste 16 bits sorteren en daarna op de hoogste 16 bits.

Bucket Sort

Bucket Sort werkt wanneer de keys ongeveer uniform verdeeld zijn over een bekend interval.

Idee:

1. Verdeel de keyruimte in n buckets.
2. Plaats elk element in de juiste bucket.
3. Sorteert iedere bucket afzonderlijk (meestal met Insertion Sort).
4. Plak alle buckets achter elkaar.

Eigenschappen:

- Verwachte tijd: $O(n)$
- Extra geheugen: $O(n)$
- Vereist uniforme verdeling van de input
- Slechtste geval: $O(n^2)$ wanneer veel elementen in dezelfde bucket terechtkomen

Ondergrens voor sorteren

Voor algoritmen die uitsluitend vergelijken gebruiken (*comparison sorts*) geldt:

$$\Omega(n \log n)$$

vergelijkingen in het slechtste geval.

Het bewijs gebruikt een decision tree:

- Elke vergelijking heeft slechts twee uitkomsten.
- Er zijn $n!$ mogelijke invoerpermutaties.
- Een boom met hoogte V heeft hoogstens 2^V bladeren.

Dus:

$$2^V \geq n!$$

en daarom:

$$V \geq \log(n!)$$

wat leidt tot:

$$V = \Omega(n \log n)$$

Hieruit volgt dat vergelijking-gebaseerd sorteren asymptotisch niet sneller kan zijn dan $n \log n$.

Conclusie

Algoritme	Tijd	Extra geheugen
QuickSort	$O(n \log n)$	$O(\log n)$
HeapSort	$O(n \log n)$	$O(1)$
MergeSort	$O(n \log n)$	$O(n)$
Counting Sort	$O(n + M)$	$O(n + M)$
Radix Sort	$O(d(n + M))$	afhankelijk van Counting Sort
Bucket Sort	verwacht $O(n)$	

9 | kansproblemen

Elke uitvoering heeft twee mogelijke uitkomsten:

$$S = \text{Success (succes)}, \quad F = \text{Fail (mislukking)}$$

met kansen

$$\Pr(S) = p, \quad \Pr(F) = q = 1 - p.$$

Onafhankelijkheid betekent dat de uitkomst van een poging geen invloed heeft op de volgende pogingen.

Voorbeelden van Bernoulli-experimenten

- **Examens:** een examen wordt herhaald totdat je slaagt.
- **PIN-code raden:** een dief mag maximaal drie pogingen doen voordat de kaart wordt geblokkeerd.
- **Mens-Erger-Je-Niet:** een pion mag pas worden opgezet als er een 6 wordt gegooid.
- **Munten gooien:** meerdere munten worden tegelijk opgegooid en de kop/munt-uitkomsten worden bekeken.

Er zijn twee belangrijke soorten problemen:

1. **Vast aantal experimenten** (bijvoorbeeld PIN-code raden of munten gooien).
2. **Vast aantal successen** (bijvoorbeeld wachten tot een examen is gehaald).

Bernoulli-proces met vaste lengte

Beschouw een rij van n onafhankelijke experimenten.

Een mogelijke uitkomst is bijvoorbeeld

$$SFS.$$

Door onafhankelijkheid geldt:

$$\Pr(SFS) = p \cdot q \cdot p = p^2 q.$$

Algemeen geldt:

Stelling

Als een rij U precies k successen en $n - k$ mislukkingen bevat, dan is

$$\Pr(U) = p^k q^{n-k}.$$

De kans hangt alleen af van het aantal successen en mislukkingen, niet van hun positie in de rij.

Kans op minstens één succes

De enige manier om geen enkel succes te behalen is wanneer alle n experimenten mislukken:

$$FFFF \dots F.$$

De kans daarop is

$$q^n.$$

Daarom geldt:

$$\Pr(\text{minstens één succes}) = 1 - q^n.$$

Voorbeeld Bij een examen is de kans op slagen per poging $p = 0.3$.

Na 5 pogingen is de kans dat je minstens één keer slaagt:

$$1 - (1 - 0.3)^5 = 1 - 0.7^5 \approx 0.832.$$

Dus ongeveer 83.2%.

Aantal successen in n experimenten

We willen nu de kans berekenen op precies k successen in n pogingen.

Voorbeeld met $n = 4$ en $k = 2$:

$$FFSS, \quad FSFS, \quad FSSF, \quad SFFS, \quad SF SF, \quad SSFF.$$

Er zijn dus

$$\binom{4}{2} = 6$$

mogelijke plaatsingen van de successen.

Algemeen zijn er

$$\binom{n}{k}$$

rijtjes met precies k successen.

Omdat elk van deze rijtjes kans

$$p^k q^{n-k}$$

heeft, volgt:

Stelling

$$\Pr(\text{exact } k \text{ successen}) = \binom{n}{k} p^k q^{n-k}.$$

Aangezien $q = 1 - p$:

$$\Pr(\text{exact } k \text{ successen}) = \binom{n}{k} p^k (1 - p)^{n-k}.$$

Binomiale verdeling

De kansverdeling

$$B_{n,p}(k) = \binom{n}{k} p^k (1 - p)^{n-k}$$

heet de **binomiale verdeling**.

Hierbij stelt:

- n = aantal experimenten;
- p = kans op succes;
- k = aantal behaalde successen.

Voorbeeld Een student maakt 10 oefenopgaven. Elke opgave wordt met kans 0.8 correct opgelost.

De kans op precies 8 correcte antwoorden is:

$$\Pr(X = 8) = \binom{10}{8} (0.8)^8 (0.2)^2.$$

Omdat

$$\binom{10}{8} = 45,$$

volgt:

$$\Pr(X = 8) = 45 \cdot 0.8^8 \cdot 0.2^2 \approx 0.302.$$

Dus ongeveer 30.2%.

Nuttige formules

Absorptieregel

$$k \binom{n}{k} = n \binom{n-1}{k-1}.$$

Bewijs:

$$k \binom{n}{k} = k \frac{n!}{k!(n-k)!} = n \frac{(n-1)!}{(k-1)!((n-1)-(k-1))!} = n \binom{n-1}{k-1}.$$

Binomium van Newton

Voor alle $n \geq 0$ geldt:

$$(x+y)^n = \sum_{k=0}^n \binom{n}{k} x^{n-k} y^k.$$

Voorbeeld:

$$(x+y)^3 = x^3 + 3x^2y + 3xy^2 + y^3.$$

De binomiale verdeling dankt haar naam aan deze formule.

10 | Abstracte datastructuren

Een **abstracte datastructuur** (ADS) beschrijft *wat* een datastructuur kan (welke methoden beschikbaar zijn en hun complexiteit), zonder te specificeren *hoe* dit intern werkt. De fysieke datastructuur (FDS) beschrijft de daadwerkelijke implementatie. Dit onderscheid maakt het mogelijk het gebruik van een datastructuur te scheiden van de implementatie ervan.

Statische en dynamische structuren

Implementaties vallen uiteen in twee categorieën:

- **Statisch** (arrays): vaste omvang, directe toegang in $O(1)$, maar duur bij invoegen op een willekeurige positie.
- **Dynamisch** (pointers/nodes): flexibele omvang, goedkoop invoegen halverwege, maar toegang tot element i kost $\Theta(i)$ tijd.

Veelgebruikte structuren in C#

- **List<T>**: intern waarschijnlijk een array. Add is gemiddeld $O(1)$ (door verdubbeling), Remove is duur, directe toegang via index is $O(1)$.
- **LinkedList<T>**: intern een dubbel gelinkte lijst. Invoegen en verwijderen bij een bekende node is $O(1)$, maar toegang via index kost $\Theta(i)$. Enumeratie moet via pointers, niet via index.
- **Dictionary<K,V>**: slaat sleutel-waardeparen op. Add, Remove en ContainsKey zijn $O(1)$, maar met een hogere impliciete constante dan List. Gebruik alleen als snel zoeken op sleutel nodig is.

Enumeratie

Elke datastructuur biedt een **Enumerator** met **Current** en **MoveNext**, zodat alle elementen op een uniforme manier doorlopen kunnen worden. In C# is dit beschikbaar via de **foreach**-lus. Let op: de verzameling mag niet worden aangepast binnen de body van een **foreach**.

Verborgen complexiteit

Enkele valkuilen:

- Stringconcatenatie met `+` in een lus heeft kwadratische complexiteit in de totale stringlengte; gebruik in plaats daarvan directe uitvoer of `StringBuilder`.
- `Remove` op een `List` en toegang via index op een `LinkedList` zijn duur.

Subset van een klein universum

Als de elementen een vaste, kleine verzameling vormen, kan een deelverzameling efficiënt worden bijgehouden als een bitrij (bitmasker). Een element i toevoegen gaat met een bitsgewijze OR: `fruitmand |= (1 << i)`; aanwezigheid controleren met `(fruitmand & (1 << i)) > 0`.

11 | Stack en Queue

De stack

Een stack kan je het beste vergelijken met een toiletpapierhouder, en een item in de stack met een toilet rol. Je houder is aan de onderkant afgesloten, zodat er niks uit valt, en je kan altijd alleen de bovenste rol van de houder pakken.

Statische stack

Stel je hebt een array met n -lengte (`S`), dan is het de bedoeling dat bij een stack items steeds achteraan worden toegevoegd, dit gebeurt met de `S.Push(x)` methode, je push namelijk iets op de stack. Dan heb je ook nog de `S.Pop()` methode die het bovenste element pakt en de waarde hiervan terug geeft.

De code van push is simpel `S[c++] = x`, c is hier de grootte van de array. Let op in C# wordt de array index eerst uitgevoerd en daarna pas de `+= 1`. Dus als $c = 0$, dan wordt eerst de waarde x op plaats 0 gezet en daarna $c += 1$ uitgevoerd.

Omgekeerd is de pop ook heel simpel `Return S[--c]`, maar hier is de order of operation eers $c -= 1$, daarna de index (omdat de `--` vóór de c staat). Dit is omdat de count aangeeft hoeveel items er zijn, bijvoorbeeld 6, dan zitten die waarde op index 0 tot en met 5, dus de laatste is `count - 1`.

Dynamische stack

Bij een dynamische stack hebben we geen array meer, maar een heleboel objecten die niet naast elkaar staan in het geheugen. Elk object heeft 2 variabelen, een key naar het vorige (oudere) element en de waarde.

Als je een element toevoegd met `S.Push(x)`, dan maak je een nieuw object aan in de linked list en dat object krijg de key van het voorheen nieuwste object. Het nieuwste object wordt door de linked list bijgehouden.

Als je `S.Pop()` aanroept, dan kijkt de linked list naar het nieuwste element, verwijdert deze en schrijft vervolgens de bijgehouden key naar het nieuwste element over naar de key die dit object heeft (wat het een-na-nieuwste object was).

Queue

Een queue kan je het beste vergelijken met een wachtrij (goh!) je sluit achteraan aan en de voorste wordt als eerste geholpen.

Statische queue

Als je een element toevoegd aan een Queue met `Q.Enqueue(x)`, dan wordt de waarde op de eerstvolgende lege plek in een array toegevoegd (wat je weet door de lengte bij te houden). Wil je een item wegwerken, dan kan je dat doen door `Q.Dequeue()` te doen, deze geeft de oudste waarde terug, dat is de eerste waarde in de array.

`Q.Enqueue(x)` wordt gedefinieerd door `Q[t++] = x`, redelijk simpel. En `Q.Dequeue()` wordt gedefinieerd door `return Q[h++]`. h en t worden apart bijgehouden, t

houdt bij waar de “tail” is in de array, dus de eerstvolgende lege plek in de array. h is de plek waar de de oudste waarde is. Wedereom betekend de `t++` en de `h++` dat eerste de index op plek t of h gepakt wordt en daarna pas de `+= 1` wordt uitgevoerd.

Dynamische queue

Bij een dynamische queue zijn er weer verschillende objecten, en elk object houdt de waarde bij en de waarde van het volgende object in de wachtrij. De `LinkedList` houdt bij wat het laatste toegevoegde item is (net zoals

bij de stack) en als je `Q.Enq(X)` aanroep, dan wordt bij de het nieuwst element (achteraan) de key naar het volgende element gezet naar de nieuw toegevoegde waarden. bij een `Q.Deq()` wordt het oudste element gepakt (bijgehouden door `h`) en wordt `h` gezet naar het een-na-oudste element.

Priority queue

Een priority queue lijkt op een gewone queue, alleen bepaalt niet de leeftijd van een item wanneer het aan de beurt is, maar de prioriteit (de key). Bij een min-priority queue wordt altijd het element met de kleinste key als eerste teruggegeven.

De belangrijkste methodes zijn:

- `Q.Insert(k)` voeg een element met key `k` toe.
- `Q.ExtractMin()` geef en verwijder het element met de kleinste key.
- `Q.Min()` bekijk het kleinste element zonder het weg te halen.

Een priority queue wordt vaak gebruikt wanneer gebeurtenissen op tijd uitgevoerd moeten worden. Bijvoorbeeld in een simulatie waarbij steeds de eerstvolgende gebeurtenis uitgevoerd moet worden. Dan krijgt elke gebeurtenis als key het tijdstip waarop deze moet plaatsvinden.

Statische priority queue

Een statische priority queue wordt meestal geïmplementeerd met een heap in een array. Hierbij staat niet alles volledig gesorteerd, want dan zou toevoegen duur worden. In plaats daarvan wordt een gedeeltelijke ordening bijgehouden.

De heap wordt opgeslagen in een array die vanaf index 1 wordt gebruikt. Als een element op positie `i` staat, dan geldt:

- Linkerkind: `2i`
- Rechterkind: `2i+1`
- Ouder: `[i/2]`

Bij een min-heap geldt dat iedere ouder kleiner of gelijk moet zijn aan zijn kinderen. Daardoor staat de kleinste key altijd op positie 1.

Een element toevoegen met `Q.Insert(k)` gebeurt door de waarde achteraan de heap te plaatsen en vervolgens omhoog te schuiven totdat de heap-eigenschap weer klopt. Dit noemen we *Rootify* of *Bubble Up*.

Een element verwijderen met `Q.ExtractMin()` gebeurt door het element op positie 1 terug te geven. Vervolgens wordt het laatste element uit de heap naar positie 1 verplaatst en met *Min-Heapify* of *Bubble Down* naar beneden geschoven totdat de heap weer correct is.

Zowel `Insert` als `ExtractMin` hebben hierdoor een looptijd van $O(\log n)$.

Dynamische priority queue

Bij een dynamische priority queue wordt dezelfde heapstructuur gebruikt, maar de elementen hoeven niet meer naast elkaar in een array te staan. In plaats daarvan kunnen afzonderlijke objecten gebruikt worden die via pointers met elkaar verbonden zijn.

Conceptueel blijft het idee hetzelfde: het element met de hoogste prioriteit staat bovenaan de boomstructuur. Bij een `Insert(k)` wordt een nieuw element toegevoegd en omhoog geschoven totdat de juiste plek gevonden is. Bij een `ExtractMin()` wordt de wortel verwijderd en wordt de heap opnieuw hersteld.

Hoewel dit mogelijk is, wordt in de praktijk meestal de statische array-implementatie gebruikt omdat de indexberekeningen van een heap erg efficiënt zijn en weinig geheugen kosten.

12 | Heap

Een heap is een datastructuur die gebruikt wordt om een priority queue efficiënt te implementeren. De heap wordt meestal opgeslagen in een array, maar wordt in gedachten gezien als een volledige binaire boom.

Bij een min-heap geldt steeds dat iedere ouder kleiner of gelijk is aan zijn kinderen. Daardoor staat het kleinste element altijd bovenaan de boom, op positie 1 van de array.

Heapify

Soms klopt de heap-eigenschap niet meer, bijvoorbeeld wanneer een grote waarde boven kleinere kinderen terecht komt. Om dit te repareren gebruiken we `MinHeapify(i)`.

Deze methode vergelijkt een element met zijn kinderen. Als één van de kinderen kleiner is, wordt gewisseld met het kleinste kind. Vervolgens wordt hetzelfde proces herhaald op de nieuwe positie van het element.

Dit wordt ook wel *Bubble Down* genoemd omdat het element steeds verder naar beneden zakt.

Omdat er maximaal door de hoogte van de boom gelopen wordt, kost `MinHeapify` $O(\log n)$.

BuildHeap

Met `BuildHeap()` kan van een willekeurige array een geldige heap gemaakt worden.

Hiervoor wordt vanaf het laatste interne knooppunt terug naar boven gewerkt:

```
for(i = n/2; i >= 1; i-) Heapify(i)
```

Bladeren hebben geen kinderen en voldoen automatisch aan de heap-eigenschap. Door vervolgens alle ouders van onder naar boven te repareren ontstaat uiteindelijk een geldige heap.

Opvallend genoeg kost dit geen $O(n \log n)$, maar slechts $O(n)$.

ExtractMin

Bij `ExtractMin()` wordt het element op positie 1 teruggegeven.

Vervolgens wordt het laatste element van de heap naar de wortel verplaatst en wordt `Heapify(1)` uitgevoerd om de heap weer te herstellen.

Omdat alleen langs één pad naar beneden gelopen wordt, kost dit $O(\log n)$.

Rootify

Het tegenovergestelde van Heapify is `Rootify(i)`.

Deze methode wordt gebruikt wanneer een element mogelijk te klein is geworden ten opzichte van zijn ouder. Het element wordt dan steeds met zijn ouder verwisseld totdat de heap-eigenschap weer klopt.

Dit wordt ook wel *Bubble Up* genoemd.

Ook hier wordt maximaal door de hoogte van de boom gelopen, dus de looptijd is $O(\log n)$.

Insert

Bij `Insert(k)` wordt eerst de heap met één element vergroot.

De nieuwe key wordt achteraan geplaatst:

```
A[++n] = k
```

Daarna wordt `Rootify(n)` uitgevoerd zodat het nieuwe element naar boven kan bewegen totdat de heap weer geldig is.

De totale looptijd is opnieuw $O(\log n)$.

DecreaseKey

Soms wil je een bestaande key kleiner maken, bijvoorbeeld in Dijkstra's algoritme.

Met `DecreaseKey(i,k)` wordt de waarde op positie i verlaagd. Hierdoor kan het zijn dat het element kleiner wordt dan zijn ouder.

De heap wordt dan hersteld met `Rootify(i)`.

Ook deze operatie kost $O(\log n)$.

Heapsort

Een heap kan gebruikt worden om te sorteren.

Eerst wordt met `BuildHeap()` een heap gemaakt van de array. Daarna wordt steeds het minimum verwijderd met `ExtractMin()` en achteraan de array geplaatst.

Dit levert een sorteeralgoritme op met een looptijd van $O(n \log n)$.

Heapsort is theoretisch even snel als QuickSort, maar in de praktijk vaak iets langzamer omdat het geheugen minder lokaal gebruikt wordt.

13 | Hashtabel

Een **Dictionary** (ADS) slaat key-value-pairs (KVP's) op en biedt snelle toegang via de key. De drie kernoperaties zijn **Insert(k,v)**, **Delete(k)** en **Search(k)**.

Directe adressering

Als alle keys uit een kleine verzameling $U = \{0, \dots, m-1\}$ komen, gebruik dan gewoon een array van lengte m met pointers naar de data. Insert, Delete en Search kosten dan $O(1)$; geheugengebruik is $\Theta(m)$, ongeacht het werkelijke aantal keys n .

Hashing

Wanneer U te groot is voor een directe array, gebruik je een **hashfunctie**:

$$H : U \rightarrow \{0, \dots, m-1\}$$

Deze kent elke key k een positie $H(k) < m$ toe. De **load factor** $\alpha = n/m$ geeft het gemiddeld aantal keys per slot aan; houd α klein.

Botsingen (meerdere keys met dezelfde hashwaarde) los je op met:

- **Chaining** (standaard in C#): elk slot bevat een lijst van KVP's. Insert, Delete en Search kosten gemiddeld $\Theta(1 + \alpha)$.
- **Open adressering**: alle KVP's staan in de array zelf; bij een botsing wordt een alternatieve locatie opgezocht via een *probe sequence*. Vereist $\alpha < 1$, deletes zijn lastig (vervang door sentinel DEL). Verwachte zoektijd voor een leeg slot: $\frac{1}{1-\alpha}$.

Eigenschappen van een goede hashfunctie

- **Deterministisch**: dezelfde key levert altijd hetzelfde resultaat.
- **Snel**: H wordt vaak aangeroepen.
- **Uniform**: $\Pr(H(x) = i) = \frac{1}{m}$ voor een willekeurige key x .
- **Universeel**: voor $x \neq y$ geldt $\Pr(H(x) = H(y)) \leq \frac{1}{m}$.

Een voorbeeld van een familie van universele hashfuncties (voor keys in $\{0, \dots, p-1\}$, p priemgetal):

$$h_{a,b}(k) = ((a \cdot k + b) \bmod p) \bmod m$$

met willekeurige $a \in \{1, \dots, p-1\}$ en $b \in \{0, \dots, p-1\}$.

Vergelijking: hashtabel vs. zoekboom

Operatie	Hashtabel	Zoekboom
Insert / Delete / Search	$O(1)$ gem.	$O(\lg n)$
Min / Max	niet ondersteund	$O(\lg n)$
Successor / Predecessor	niet ondersteund	$O(\lg n)$
Enumereren op volgorde	niet ondersteund	$O(n)$

In C#: `Dictionary<K,V>` gebruikt hashing; `SortedDictionary<K,V>` gebruikt een zoekboom.

14 | Binaire Zoekbomen

Een **binaire zoekboom** (BZB) is een binaire boom waarin elke knoop een key bevat, zodanig dat:

- alle keys in de *linker* deelboom van knoop x zijn $\leq x.key$,
- alle keys in de *rechter* deelboom van knoop x zijn $\geq x.key$.

Dit heet de **BZB-eigenschap**. Alle operaties kosten $O(h)$, met h de hoogte van de boom. Voor een gebalanceerde boom is $h = \Theta(\lg n)$; in het slechtste geval (volledig scheef) is $h = \Theta(n)$.

Begrippen:

- **Diepte** van knoop x : lengte van het pad van de wortel naar x .
- **Hoogte** van knoop x : lengte van het langste pad van x naar een blad.
- **Hoogte van de boom**: hoogte van de wortel.

Inorder-doorloop

Om alle keys gesorteerd te printen gebruik je een *inorder*-doorloop:

```
Inorder-Tree-Walk(x):
    if x != nil:
        Inorder-Tree-Walk(x.left)
        print x.key
        Inorder-Tree-Walk(x.right)
```

Aanroep: `Inorder-Tree-Walk(T.root)`. Complexiteit: $\Theta(n)$.

Zoeken

```
Tree-Search(x, k):
    if x == nil or k == x.key: return x
    if k < x.key: return Tree-Search(x.left, k)
    else:         return Tree-Search(x.right, k)
```

Iteratief equivalent loopt langs de boom omlaag totdat de key gevonden is of een `nil`-pointer bereikt wordt. Complexiteit: $O(h)$.

Minimum, Maximum, Successor en Predecessor

- **Minimum**: volg steeds `x.left` totdat `nil`. Complexiteit $O(h)$.
- **Maximum**: volg steeds `x.right` totdat `nil`. Complexiteit $O(h)$.
- **Successor** van x (de volgende key in inorder-volgorde):
 - Als x een rechter kind heeft: `Tree-Minimum(x.right)`.
 - Anders: volg ouder-pointers omhoog totdat een voorouder het *linker*kind is van zijn ouder; die ouder is de opvolger.
- **Predecessor**: spiegelbeeld van Successor.

Alle vier kosten $O(h)$.

Toevoegen

Zoek de juiste lege plek door de boom omlaag te volgen (links als $z.key < x.key$, anders rechts) en voeg het nieuwe element daar in. De BZB-eigenschap blijft behouden. Complexiteit: $O(h)$.

```
Tree-Insert(T, z):
    y = nil; x = T.root
    while x != nil:
        y = x
        if z.key < x.key: x = x.left
        else:             x = x.right
    z.p = y
    if y == nil:          T.root = z          // lege boom
    elif z.key < y.key:    y.left = z
    else:                 y.right = z
```

Verwijderen

Verwijder knoop z op basis van drie gevallen:

1. z **heeft geen kinderen**: vervang z door `nil`.
2. z **heeft één kind**: schuif dat kind omhoog naar de plek van z .

3. **z heeft twee kinderen:** zoek de opvolger y van z (het minimum van de rechter deelboom van z ; y heeft zelf geen linker kind). Vervang y door zijn rechter kind, zet y vervolgens op de plek van z .

Hulpfunctie `Transplant(T, u, v)` vervangt deelboom met wortel u door deelboom met wortel v . Complexiteit van verwijderen: $O(h)$.